Phase 3 - Marc Petrosyan, Vahan Zakaryan, Sergey Hayriyan

Table	Columns	Rows
user	id, username, email, password_hash, avatar_url, karma, created_at	2100
group	id, created_by, name, description, banner_url, member_count, is_nsfw, created_at	1050
follow	user_id, subreddit_id, subscribed_at	6300
moderator	user_id, subreddit_id, permissions, added_at	1050
post	id, author_id, subreddit_id, post_type, title, body, url, score, comment_count, is_archived, created_at	7500
comment	id, post_id, author_id, parent_comment_id, body, score, is_archived, created_at	22000
vote	id, user_id, target_id, target_type, value, created_at	26500
tag	id, subreddit_id, text, color	1500
post_tag	post_id, tag_id, assigned_at	4200
report	id, reporter_id, resolved_by, target_id, target_type, reason, status, created_at, resolved_at	1250
notification	id, user_id, actor_id, target_id, type, target_type, is_read, created_at	13000
Total		86450

Data Collection Strategy

Synthetic Generation

The majority of data is generated using Python with the Faker library to generate 11 csv files (we'll use the COPY command)

- User data: usernames and emails generated via Faker; karma values drawn from a log-normal distribution to simulate real skew.
- Group data: group names drawn from a curated list of realistic topic domains (technology, sports, science, gaming, etc.).
- Posts/Comments: Body text with lorem ipsum. Post titles modelled after real title patterns (question, statement, link-share).
- Votes: Generated with a strong upvote bias (90% upvotes) to mimic Reddit.
- Timestamps: Distributed over a simulated 2-year window.

Enum Distribution

Enum Column	Values	Approximate Distribution
vote.target_type	post, comment	40% post / 60% comment
post.post_type	text, link, image	55% / 25% / 20%
report.status	pending, approved, dismissed	60% / 25% / 15% accordingly
report.target_type	post, comment	50% / 50%
notification.type	comment_reply, post_comment, mention, award_received, mod_action	45% / 30% / 15% / 8% / 2% accordingly
notification.target_type	post, comment, user	35% / 50% / 15% accordingly

Operational Processes

User Registration

When a new user signs up, a record is inserted into the user table with a generated UUID, chosen username, hashed password, and initial karma of 0. The system must first verify that the username and email are unique before committing the insert. The created_at timestamp is set automatically via a default value or trigger.

- Tables involved: user
- Operation type: INSERT with uniqueness pre-check (SELECT)
- Business rule: Duplicate username or email must be rejected.

Post Submission

A logged-in user creates a post inside a group (subreddit). The system inserts a row into post referencing the author and the subreddit. Initial score and comment_count are set to 0. A trigger automatically creates the author's own upvote (value = 1) in the vote table upon each new post, in line with Reddit convention.

- Tables involved: post, vote
- Operation type: INSERT (post), trigger-driven INSERT (vote)
- Business rule: Only members of a group may post; NSFW posts inherit the flag.

Voting on Posts and Comments

A user casts a vote (upvote = 1, downvote = −1) on a post or comment. The target_type discriminator (enum vote_target) determines whether target_id resolves to a post or a comment. The score of the target entity must be updated atomically. If the same user votes again on the same target, the existing vote row is updated (vote changed) or deleted (vote rescinded). A trigger maintains the denormalised score column on post and comment.

- Tables involved: vote, post, comment
- Operation type: INSERT / UPDATE / DELETE on vote; trigger UPDATE on post or comment
- Business rule: One vote per user per target; score = SUM of vote values.

Posting a Comment

A user submits a comment on a post. If it is a top-level reply the parent_comment_id is NULL; for nested replies it references an existing comment within the same post. The comment_count on the parent post is incremented by a trigger. A notification of type comment_reply or post_comment is generated for the relevant recipient.

- Tables involved: comment, post, notification
- Operation type: INSERT (comment); trigger UPDATE (post.comment_count); trigger INSERT (notification)
- Business rule: Archived posts (is_archived = TRUE) do not accept new comments.

Following / Unsubscribing from a Group

A user subscribes to a group by inserting a row into follow. Unsubscribing removes that row. The denormalised member_count on group must be kept consistent via triggers on INSERT and DELETE of follow.

- Tables involved: follow, group
- Operation type: INSERT / DELETE on follow; trigger UPDATE on group.member_count
- Business rule: A user may not follow the same group twice (unique constraint on composite PK).

Content Reporting & Moderation

Any user can file a report against a post or comment using the report table. The target_type discriminator (enum report_target) identifies whether the report targets a post or comment. Reports begin with status pending. A moderator of the relevant group resolves the report by updating status to either approved (content removed) or dismissed, and setting resolved_by and resolved_at.

- Tables involved: report, moderator, post, comment
- Operation type: INSERT (report); UPDATE (report.status, resolved_by, resolved_at)
- Business rule: Only moderators of the targeted group may resolve reports.

Moderator Assignment

The creator of a group (or an existing moderator with sufficient permissions) can promote a user to moderator by inserting into moderator.

- Tables involved: moderator, group, user
- Operation type: INSERT / DELETE
- Business rule: A group must always retain at least one moderator.

Tag Assignment to Posts

Each group may define its own set of tags (tag table). When a post is submitted, authors can select one or more applicable tags, creating rows in post_tag. The system validates that the chosen tag belongs to the same subreddit as the post before inserting.

- Tables involved: post_tag, tag, post
- Operation type: INSERT with foreign-key + business validation
- Business rule: Tags must belong to the same group as the post

Analytical & Reporting Queries

Top Contributors by Karma

Returns the 10 users with the highest cumulative karma scores, alongside their total post count, comment count, and average post score. We're planning to use this for a "verified contributor" badge system, the idea is that high-karma users get a small indicator on their profile that signals trust to newer members. The aggregation is straightforward but the join across three tables (user, post, comment) means we'll want to check the execution plan once the data is populated.

Tables	Key Technique
user, post, comment	**Aggregation, ORDER BY, LIMIT**

Most Active Groups (Last 30 Days)

Ranks subreddits by number of new posts in the past 30 days, with moderator count and follower count included. The editorial team needs this to decide which communities to feature on the front page, which groups are actually alive right now versus which ones had a surge six months ago and went quiet.

Tables	Key Technique
group, post, follow, moderator	**JOIN, GROUP BY, date filtering**

Trending Posts (Hot Score)

Computes a hot score for each post using the formula: score / $(\text{age_in_hours} + 2)^{1.8}$, then returns the top 25 posts from the last 48 hours. This feeds directly into the front-page ranking. The exponent 1.8 is borrowed from Reddit's original algorithm.

Tables	Key Technique
post, group	**Computed expression, ORDER BY**

Engagement Rate per Group

For each group, computes the ratio of total comments to total posts over the last 90 days. Groups with fewer than 5 posts in that window are excluded, otherwise a group with 1 post and 3 comments looks like a 300% engagement rate, which is misleading.

Tables	Key Technique
group, post, comment	HAVING, ratio aggregation

Users with No Activity in 90 Days

Find users who have not authored any post or comment in the past 90 days and whose account is older than 30 days. Will target inactive users with re-engagement emails.

Tables	Key Technique
user, post, comment	NOT EXISTS / LEFT JOIN … IS NULL

Report Resolution Time Report

For each moderator computes the average time (in hours) between report creation and resolution, and the total number of reports handled, broken down by month. Will be used for moderation team performance review.

Tables	Key Technique
report, moderator, user	EXTRACT, GROUP BY month

Most Reported Content (Pending)

List the top 20 posts and comments by number of pending reports, joined to the group they belong to. Needed for creating a priority queue which will be useful for moderation review.

Tables	Key Technique
report, post, comment, group	UNION / subquery, ORDER BY

Notification Read Rate by Type

For each notification_type, compute the percentage of notifications that have been read (is_read = TRUE). Can be used to evaluate which notification types drive re-engagement.

Tables	Key Technique
notification	CASE, ratio aggregation

Co-followers Recommendation

For a given user U, returns other users who follow at least 3 of the same groups as U but whom U doesn't follow yet. This is a lightweight "people you may know" feature that only uses data we already have. The threshold of 3 shared groups is somewhat arbitrary, we might lower it to 2 after seeing the data, since 3 might return almost no results if the average user follows fewer than 5 groups.

Tables	Key Technique
follow, user	Self-Join, COUNT, NOT IN

Tag Usage Frequency per Group

For each group, it ranks its tags by how many posts they've been applied to in the last 60 days. The practical use is tag curation: group moderators should periodically retire tags that no one uses and promote the ones people actually reach for. Without this query there's no way to tell which tags are just clutter.

Tables	Key Technique
tag, post_tag, post, group	Window function RANK()

JoinQuerySpecifications

Join type: INNER JOIN

Retrieve all posts together with their author username and the group name. Only posts where both the author and group still exist are returned. Used for the standard content feed query.

```
post INNER JOIN user on post.author_id = user.id
INNER JOIN group ON post.subreddit_id = group.id
```

Join type: LEFT OUTER JOIN

List all users and, for each, the number of posts they have authored. Users with zero posts must still appear (LEFT JOIN ensures NULLs are included). Used for the contributor leaderboard so that newly registered users appear with a post count of 0.

```
user LEFT JOIN post ON user.id = post.author_id
GROUP BY user.id
```

Join type: RIGHT JOIN

List all groups and the number of active moderators, including groups with no current moderators (returned via NULL on the user side). Business use: detect unmaintained communities.

```
moderator RIGHT JOIN group ON moderator.subreddit_id = groupd.id
GROUP BY group.id
```

Join type: FULL JOIN

Cross-audit of users and groups: return all users and all groups, matching where the user is the group creator, and surfacing both users who have never created a group AND groups whose creator has been deleted. Business use: data integrity audit.

```
user FULL JOIN group ON user.id = group.created_by
```

Join type: SELF JOIN

Retrieve comment threads by joining the comment table to itself on parent_comment_id = id, enabling reconstruction of full nested reply chains. Business use: threaded comment rendering.

```
comment AS child
JOIN comment AS parent ON child.parent_comment_id = parent_id
```

Join type: CROSS JOIN (limited)

Generate all tag-post combinations within a given subreddit for a tag-recommendation matrix. Applied after filtering to a small working set to avoid full Cartesian explosion. Business use: auto-suggest relevant tags.

```
post CROSS JOIN tag
WHERE post.subreddit_id = tag.subreddit_id
AND post.id NOT IN (SELECT post_id FROM post_tag WHERE tag_id = tag.id)
```

Subquery Specifications

Scalar Subquery (in SELECT clause)

For each post in the feed, display the total number of comments inline by embedding a COUNT subquery in the SELECT list. This is used instead of the denormalised comment_count to verify data consistency.

```
SELECT p.title,
        (SELECT COUNT(*) FROM comment c WHERE c.post.id = p.id) AS
live_comment_count
FROM post p
```

Correlated Subquery (in WHERE clause)

Find all users who have cast at least one downvote (value = -1) in the past 7 days. The subquery references the outer user row for each iteration. Business use: identify potentially hostile voting behaviour.

```
SELECT u.username FROM user u
WHERE EXISTS (
        SELECT 1 FROM vote v
        WHERE v.user_id = u.id
                AND v.value = -1
                AND v.created_at >= NOW() - INTERVAL '7 days'
```

IN Subquery

Retrieve all posts that belong to groups flagged as NSFW so they can be excluded from non-authenticated or safety-restricted feeds.

```
SELECT * FROM post
WHERE subreddit_id IN (SELECT id FROM group WHERE is_nsfw = TRUE)
```

NOT IN Subquery

Find users who have never filed any report. These users may be prioritised for trust-level upgrades since they have not been involved in moderation disputes.

```
SELECT id, username FROM user
WHERE id NOT IN (SELECT reporter_id FROM report)
```

Derived Table Subquery (in FROM clause)

Compute the average karma of the top-10% of users and compare it against the platform average. The inner query ranks users; the outer query aggregates only the top decile. Business use: VIP user tier threshold calibration.

```
SELECT AVG(sub.karma) AS top10_avg_karma FROM (
        SELECT karma FROM user
        ORDER BY karma DESC
        LIMIT (SELECT COUNT(*)/10 FROM user)
        ) AS sub
```

EXISTS Subquery

Return all groups that currently have at least one pending report against any of their posts. Business use: moderator alert dashboard.

```
SELECT g.name FROM group g
WHERE EXISTS (
        SELECT 1 FROM post p
        JOIN report r ON r.target_id = p.id
        WHERE p.subreddit_id = g.id AND r.status = 'pending'
)
```

Stored Functions & Procedures

Function: get_post_hot_score(post_id UUID) → FLOAT

Calculates the hot score for a given post using its score and age. Returns a float value representing ranking priority. Used by the front-page feed query to avoid re-computing the formula in application code.

```
FUNCTION get_post_hot_score (p_id UUID)
RETURNS FLOAT AS $$
    DECLARE
        v_score INTEGER;
        v_age_h FLOAT;
        v_sign INTEGER;
    BEGIN
        SELECT score, EXTRACT (EPOCH FROM
        NOW () -created_at)/3600 INTO v_score, v_age_h
        FROM post WHERE id = p_id;
        v_sign := CASE WHEN v_score> 0 THEN 1
                    WHEN v_score < 0 THEN -1 ELSE 0 END;
        RETURN v_sign * LOG(GREATEST(ABS(v_score), 1))
            + (v_age_h / (v_age_h + 2)^1.8);
    END;
$$ LANGUAGE plpgsql STABLE;
```

Function: get_user_karma(user_id UUID) → INTEGER

Computes a user's current karma as the sum of all scores on their posts and comments. Called during profile rendering or leaderboard generation. Returns the live (non-denormalised) total, useful for consistency checks.

```
FUNCTION get_user_karma (u_id UUID)
RETURNS INTEGER AS $$
    SELECT COALESCE(SUM(score), 0)
    FROM (
        SELECT score FROM post WHERE author_id = u_id
        UNION ALL
        SELECT score FROM comment WHERE author_id = u_id
        ) AS scores;
$$ LANGUAGE sql STABLE;
```

Procedure: archive_old_content(cutoff_days INTEGER)

Marks all posts and their comments as archived if the post was created more than *cutoff_days* days ago and has fewer than 5 comments. This is a scheduled maintenance procedure run nightly to freeze stale content and reduce write load on old rows.

```
PROCEDURE archive_old_content (p_days INTEGER)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE post SET is_archived = TRUE
    WHERE created_at < NOW() - (p_days || 'days') :: INTERVAL
        AND comment_count < 5
        AND is_archived = FALSE;
    UPDATE comment SET is_archived = TRUE
    WHERE post_id IN (
        SELECT id FROM post WHERE is_archived = TRUE)
        AND is_archived = FALSE;
END;
$$;
```

Procedure: resolve_report(report_id UUID, mod_id UUID, new_status report_status)

Validates that the resolving user is a moderator of the group containing the reported content, then updates the report row. If the status is approved, the procedure also archives the offending post or comment. Business need: encapsulate moderation business logic in one atomic unit.

```
PROCEDURE resolve_report (p_report UUID, p_mod UUID, p_status report_status)
LANGUAGE plpgsql AS $$
DECLARE v_target UUID; v_type report_target; v_group UUID;
BEGIN
        SELECT target_id, target_type INTO v_target, v_type
        FROM report WHERE id = p_report;
        -- derive group from target
        SELECT subreddit_id INTO v_group FROM post WHERE id = v_target;
        -- validate moderator
        IF NOT EXISTS (SELECT 1 FROM moderator WHERE user_id = p_mod
                                AND subreddit_id = v_group) THEN
                RAISE EXCEPTION 'User is not a moderator of this group';
        END IF;
        UPDATE report SET status = p_status, resolved_by = p_mod,
                resolved_at = NOW()
                WHERE id = p_report;
                IF p_status = 'approved' THEN UPDATE post SET is_archived = TRUE
                        WHERE id = v_target;
                END IF;
END;
$$;
```

Trigger Specifications

Trigger: trg_vote_update_score

Timing / Event	Tables Affected
AFTER INSERT OR UPDATE OR DELETE ON vote	**post, comment**

Maintains the denormalised score column on post and comment. When a vote is inserted, the target's score is incremented by the vote value. On update, the delta (NEW.value - OLD.value) is applied. On delete, the old vote value is subtracted. The target_type discriminator directs the UPDATE to the correct table. This avoids expensive full recalculations on every read.

Trigger: trg_comment_increment_count

Timing / Event	Tables Affected
AFTER INSERT OR DELETE ON comment	**post**

Increments post.comment_count by +1 on comment insert, and decrements by -1 on comment delete. Keeps the denormalised counter accurate without requiring a full COUNT(*) subquery on every post load. Handles cascaded deletes gracefully by operating on each affected row.

Trigger: trg_follow_member_count

Timing / Event	Tables Affected
AFTER INSERT OR DELETE ON follow	**group**

Keeps the group.member_count field synchronised. On INSERT into follow, the relevant group's member_count is incremented. On DELETE, it is decremented. This allows member count to be displayed efficiently without a live COUNT query joining follow on every group page render.

Trigger: trg_notify_on_comment

Timing / Event	Tables Affected
AFTER INSERT ON comment	**notification**

When a new top-level comment is added to a post, inserts a notification of type post_comment for the post's author. When a nested reply is added, inserts a notification of type comment_reply for the parent comment's author. The actor_id is set to the commenter's ID. Notifications are skipped if the actor and recipient are self-commenting.

Trigger: trg_prevent_archived_comment

Timing / Event	Tables Affected
BEFORE INSERT ON comment	**-**

Checks whether the post referenced by the new comment has is_archived = TRUE. If so, the trigger raises an exception and aborts the insert. This enforces the business rule that no new comments can be added to archived posts, without needing application-level enforcement.

Index Design & Performance Justification

Indexes are selected based on the access patterns described in Sections 1-2. Performance impact will be verified in Phase Four with results presented before and after index creation. All index types are standard PostgreSQL B-Tree unless otherwise noted.

Index: idx_post_subreddit_created

Type	Definition
B-Tree	**post(subreddit_id, created_at DESC)**

Business Justification: The most frequent query pattern is 'fetch recent posts for a given group'. Without this composite index, PostgreSQL performs a sequential scan of the entire post table and filters by subreddit_id, then sorts. With the index, the planner can satisfy both the equality filter on subreddit_id and the ORDER BY created_at DESC via an index scan, eliminating a sort step.

Performance Analysis Plan: Expected impact: query time reduction from O(n) seq scan to O(log n + k) index scan on large post tables.

Index: idx_comment_post_id

Type	Definition
B-Tree	**comment(post_id)**

Business Justification: Every post page load requires fetching all comments for a specific post_id. Without an index, PostgreSQL scans the entire comment table. With 20 000+ comment rows and an average of 2-3 comments per post, the ratio of comments per post is low, making index lookup dramatically faster than a full scan.

Performance Analysis Plan: Expected impact: comment-fetch query drops from full table scan to targeted index scan. Particularly important for popular posts with 100+ comments.

Index: idx_vote_user_target

Type	Definition
B-Tree	**vote(user_id, target_id, target_type)**

Business Justification: Before accepting a new vote, the system must check whether the user has already voted on the target (enforcing one-vote-per-user constraint). This lookup runs on every single vote action, making it extremely hot. The composite index covers all three columns used in the WHERE clause of this constraint check, enabling a single index probe.

Performance Analysis Plan: Expected impact: constraint-check query becomes an 'Index Only Scan'. Without this index, each vote submission requires a sequential scan of the entire vote table.

Index: idx_report_status

Type	Definition
B-Tree (partial)	**report(status) WHERE status = 'pending'**

Business Justification: Moderator dashboards query exclusively for pending reports. The total report table may hold resolved historical records (approved + dismissed) that are never queried in this context. A partial index over only pending rows is significantly smaller and faster than a full index, and allows PostgreSQL to perform an index scan that directly returns only actionable items.

Performance Analysis Plan: Expected impact: partial index size ~ 15% of a full index on status. Query time for moderator pending-report list drops proportionally.

Index: idx_notification_user_unread

Type	Definition
B-Tree (partial)	**notification(user_id, created_at DESC) WHERE is_read = FALSE**

Business Justification: The notification bell query, 'fetch unread notifications for user X, newest first', is called on every authenticated page load. It must be fast. A partial index over only unread notifications dramatically reduces the index size (most notifications are eventually read and leave the hot set).

Performance Analysis Plan: Expected impact: notification bell latency drops for active users with large notification histories.

Performance Analysis Methodology

For each index, we will present 2 EXPLAIN ANALYZE outputs - one captured before the index is created, one after. Metrics recorded will include: Planning Time (ms), Execution Time (ms), Actual Rows, Actual Loops, and the Scan node type.